UNIT 3 – CLOUD SECURITY ASSIGNMENT

SET UP A WEB APPLICATION ON AMAZON EC2

Submitted By: Arnav Wable
PRN: 2124UCSM1042

---

1. ABSTRACT

Cloud computing provides flexible and scalable infrastructure for hosting web applications over the internet. Amazon Web Services (AWS) offers Amazon Elastic Compute Cloud (EC2), which allows users to create and manage virtual servers in the cloud.

The objective of this assignment is to launch an EC2 instance, configure security rules, install a web server, and deploy a simple HTML web application. In this implementation, a Linux-based EC2 instance was used with the Nginx web server. A custom HTML page was created and hosted on the instance.

A security group was configured to allow SSH access on port 22 from the administrator's IP address and HTTP access on port 80 for public web access. The deployed website was tested using a web browser and the public IPv4 address of the EC2 instance.

This assignment demonstrates the basic process of cloud-based web hosting and the importance of network security when deploying applications on AWS.

2. INTRODUCTION

Cloud computing enables users to access computing resources such as servers, storage, databases, and networking services over the internet. Instead of maintaining physical servers, users can deploy applications using cloud platforms such as Amazon Web Services.

Amazon EC2 is a service that provides virtual servers called instances. These instances can be configured with different operating systems, storage capacities, and network settings according to application requirements.

A web server is software that receives HTTP requests from users and sends web pages in response. Nginx is a lightweight and efficient web server that can serve static HTML files and handle web traffic.

In this assignment, an EC2 instance was configured as a web server. Nginx was used to host a custom HTML page, while AWS security groups controlled access to the instance.

3. AIM

To launch an Amazon EC2 instance, configure network security, install and configure a web server, deploy a simple HTML web application, and verify its accessibility through a web browser.

4. OBJECTIVES

1. To create and launch a virtual server using Amazon EC2.
2. To configure a security group for secure network access.
3. To connect to the Linux instance using a terminal.
4. To install and configure the Nginx web server.
5. To create and deploy a custom HTML web page.
6. To verify the deployed website using its public IPv4 address.
7. To understand the role of security groups and network access controls in cloud security.

5. REQUIREMENTS

5.1 Hardware Requirements

- A laptop or desktop computer.
- A stable internet connection.

5.2 Software and Services

- Amazon Web Services account.
- Amazon EC2 service.
- Linux-based EC2 instance.
- Nginx web server.
- HTML and CSS.
- Web browser such as Google Chrome.
- Terminal or SSH connection.

6. SYSTEM ARCHITECTURE

The system consists of a client computer, AWS networking components, an EC2 virtual server, and the Nginx web server.

The user accesses the website using the public IPv4 address of the EC2 instance. The request passes through the AWS network security rules and reaches the Nginx web server. Nginx processes the request and returns the custom HTML page to the user's browser.

Architecture flow:

Client Browser → Internet → AWS Security Group → EC2 Linux Instance → Nginx Web Server → HTML Web Page

The security group allows SSH traffic on port 22 from the authorized administrator's IP address and HTTP traffic on port 80 for website access.

7. PROCEDURE

Step 1: Sign in to AWS

The AWS Management Console was opened in a web browser. The EC2 service was selected from the AWS Console search bar.

Step 2: Launch an EC2 Instance

A new EC2 instance was created with a Linux-based operating system. The instance was configured with an appropriate instance type, key pair, and network settings. A public IPv4 address was assigned to enable access over the internet.

[Insert Screenshot 2: EC2 instance showing Running state and status checks]

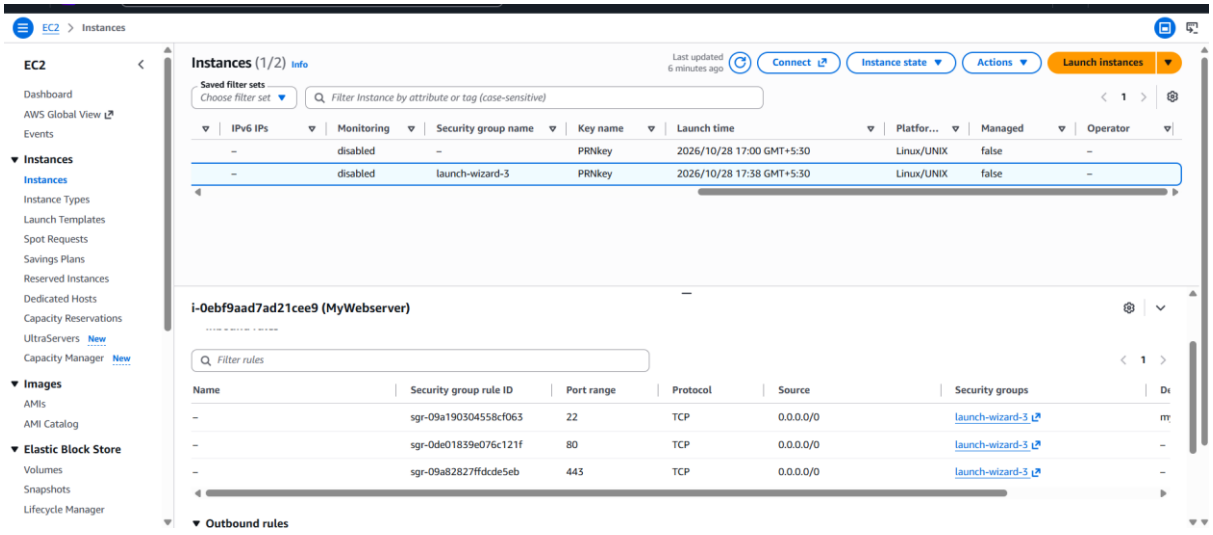
Step 3: Configure the Security Group

A security group was created to control incoming network traffic to the EC2 instance.

The following inbound rules were configured:

Type	Protocol	Port	Source
SSH	TCP	22	Administrator's public IP address
HTTP	TCP	80	0.0.0.0/0

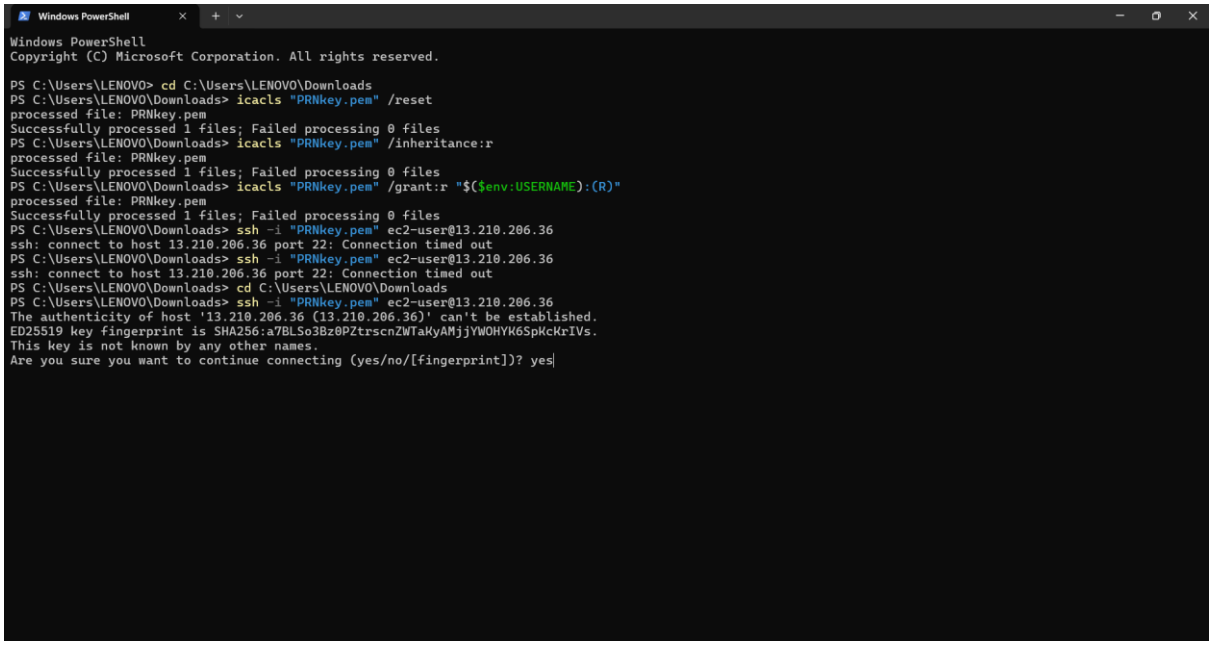
SSH access was restricted to the administrator's IP address to reduce unauthorized access. HTTP traffic was allowed from all IPv4 addresses so that users could access the website publicly.



Step 4: Connect to the EC2 Instance

The EC2 instance was accessed through a terminal connection. A Linux shell was used to execute commands and configure the web server.

The Linux terminal provided access to the instance for software installation, file management, and web server configuration.



Step 5: Install and Configure Nginx

Nginx was used as the web server. The package manager was used to install the required software.

The following commands were used where applicable:

Update package information:

sudo apt update

Install Nginx:

sudo apt install nginx -y

Start Nginx and enable it at system startup:

sudo systemctl enable --now nginx

Check the Nginx service:

sudo systemctl status nginx

These commands install Nginx, start the web server, and configure it to start automatically when the instance boots.

Step 6: Create the HTML Web Application

The default Nginx HTML page was replaced with a custom HTML page. The page included a welcome heading, deployment status, cloud security assignment title, and information about the hosting environment.

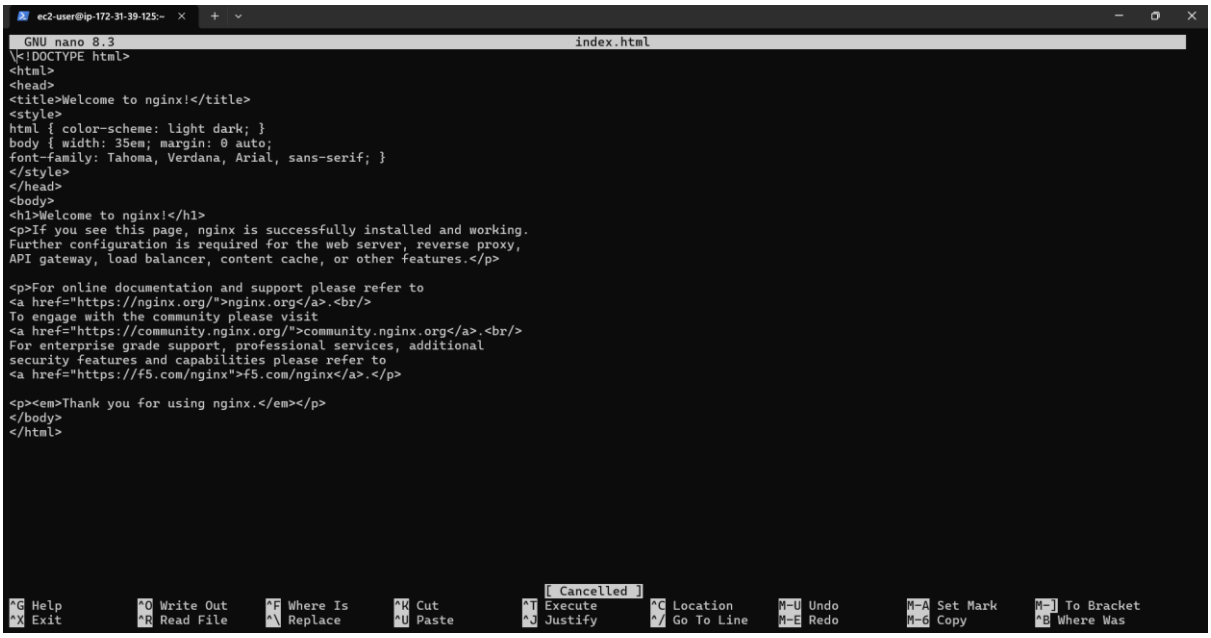
The HTML file was stored at:

/var/www/html/index.html

The Nano text editor was used to edit the file. After adding the HTML and CSS code, the file was saved.

The custom web page displayed the following information:

- Welcome to My AWS Web Application.
- Website Successfully Deployed.
- Unit 3 – Cloud Security.
- Hosted on an Amazon EC2 instance.
- Web server: Nginx.
- Basic information about the configured network security.



Step 7: Verify the Web Server

The Nginx configuration was tested using:

sudo nginx -t

If the configuration test passed, Nginx was reloaded:

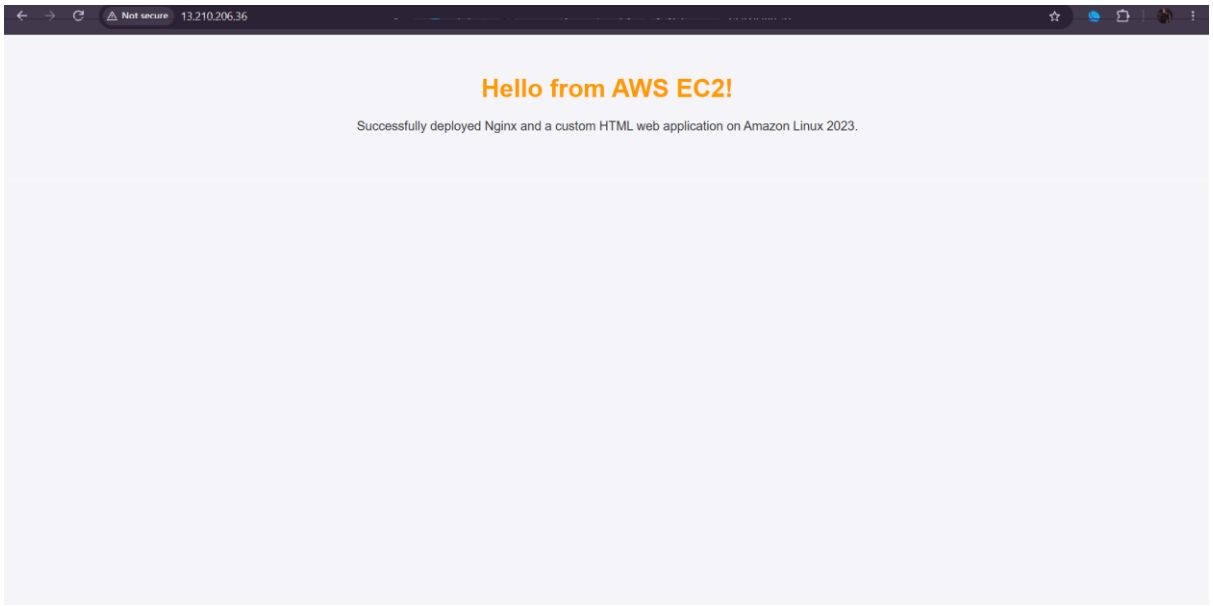
sudo systemctl reload nginx

The website was then opened in a browser using the public IPv4 address of the EC2 instance:

http://13.210.206.36

The placeholder was replaced with the actual public IPv4 address assigned to the instance.

The custom HTML page was displayed in the browser, confirming that the web server was serving the application.



8. SECURITY CONFIGURATION

Security configuration is an important part of deploying applications in the cloud. AWS security groups act as virtual firewalls that control incoming and outgoing network traffic.

The following security measures were applied:

1. SSH access was restricted to the administrator's public IP address.
2. Port 22 was used for remote terminal access.
3. Port 80 was opened to allow public HTTP requests.
4. The EC2 instance was accessed using an authorized connection method.
5. The private key file was kept private and was not shared publicly.
6. Only the required inbound ports were opened for this basic web hosting task.

The website uses HTTP, which does not encrypt traffic between the browser and the server. HTTPS with a valid TLS certificate should be configured for production use.

9. TESTING AND RESULTS

The application was tested to confirm that the EC2 instance and web server were functioning correctly.

Test	Expected Result	Actual Result
EC2 instance launch	Instance reaches Running state	[Enter your result]
Instance status checks	All required checks pass	[Enter your result]
Security group configuration	SSH and HTTP rules are present	[Enter your result]
Nginx service	Service is active	[Enter your result]
HTML file deployment	Custom HTML file is saved	[Enter your result]
Browser verification	Custom page opens using public IP	[Enter your result]

The final test confirmed successful deployment only after the custom web page was accessible in the browser. Actual results should be recorded based on the tests performed.

10. ADVANTAGES

1. Provides a simple method for hosting a website in the cloud.
2. Eliminates the need to maintain a physical web server.
3. Allows remote server administration.
4. Provides configurable network security through security groups.
5. Supports scalable cloud infrastructure.
6. Enables access to the website over the internet.

11. LIMITATIONS

1. The website uses HTTP rather than encrypted HTTPS.
2. A public web server requires appropriate security configuration and maintenance.
3. The instance and associated AWS resources may incur charges.
4. The application is a basic static HTML website and does not include a database or backend.
5. The availability of the website depends on the EC2 instance and its network configuration.

12. CONCLUSION

In this assignment, a web application was deployed on Amazon EC2 using a Linux-based instance and the Nginx web server. The security group was configured to permit SSH access from an authorized IP address and HTTP access for public website traffic.

A custom HTML page was created and placed in the Nginx web directory. The web server configuration was tested, and the application was verified through a browser using the instance's public IPv4 address.

This practical exercise provided an understanding of cloud-based web hosting, Linux server administration, web server configuration, and basic cloud network security using AWS.

13. REFERENCES

1. Amazon EC2 User Guide: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/
2. Amazon VPC User Guide: https://docs.aws.amazon.com/vpc/latest/userguide/
3. Nginx Official Documentation: https://nginx.org/en/docs/
4. Ubuntu Server Documentation: https://documentation.ubuntu.com/server/